

Socket Programming Basics

Socket Programming Basics

1. Experiment Overview
2. Core Concepts
 - 2.1 Custom Protocol Packet Structure
 - Pack Example: "Hello YAHBOOM" (13-byte payload)
 - 2.2 length Field Calculation
 - 2.3 Checksum Calculation (ICMP Style)
 - Calculation Steps (ESP32 Little-Endian View)
 - C Code Implementation
 - Pack Call Flow
 - Receive Packet Verification Flow
 - 2.4 Network Byte Order
3. Project Architecture and Flow
4. Hardware Dependencies
5. Source Code Analysis
 - 5.1 Header Files and Macro Configuration
 - 5.2 WiFi Initialization and AP Scan
 - 5.3 wifi_handler() — With AP Scan
 - 5.4 socket_experiment() — Pack/Send/Receive/Verify
 - 5.6 Key Points of Pack Flow
6. Operation Flow
 - 6.1 Build and Flash Flow
 - 6.2 Network Debugging Assistant Configuration
 - 6.3 Pack Tool
 - 6.4 Normal Log and Step Breakdown
7. Common Problems

1. Experiment Overview

Custom binary protocols are one of the most core and fundamental skills in embedded IoT development. Whether smart home devices report temperature/humidity data, industrial sensors push device status regularly, or OTA firmware upgrade package distribution and verification—all rely on custom protocol packet assembly and unpacking. Mastering this skill means you can make devices exchange binary data in the **most efficient, compact, and reliable** way, rather than depending on redundant JSON or plain text strings.

This experiment introduces **custom binary protocols** on top of TCP Socket communication, demonstrating the most core skill in embedded communication—**Packet Assembly (Pack)** and **Unpack**.

In actual IoT products, raw string transmission is inefficient, insecure, and has no verification. Custom binary protocols precisely define each frame's data format, using **Checksum** to ensure data integrity. Main application scenarios:

- **Multi-sensor data reporting:** Pack temperature, humidity, pressure and other sensor data into one binary frame, send at once, saving bandwidth and power.

- **Inter-device command interaction:** Upper computer sends control commands (on/off/speed adjust), device parses and executes, replies with acknowledgment frame—frame header + type + checksum ensures no command misjudgment.
- **OTA firmware upgrades:** Upgrade packages are numbered and packed in fixed-size chunks, each chunk with checksum, ensuring firmware files are not corrupted during transmission.
- **Private IoT protocols:** Most IoT manufacturers use custom binary protocols to connect to cloud platforms—more traffic-efficient and secure than MQTT+JSON.

Comparison	Plain String Protocol	Custom Binary Protocol
Data density	Low ("temp:25.5" = 9 bytes)	High (float 4 bytes)
Parsing complexity	Simple, string processing needed	Fixed offset direct reading
Verification capability	Usually none	ICMP-style checksum
Extensibility	String format changes need parsing updates	Adding fields only extends struct
Typical applications	Debugging, simple control	Mass production, multi-sensor data reporting

This project uses ESP32 as TCP client. After WiFi connection, scan nearby APs → connect to specified WiFi → create TCP Socket → build 72-byte custom protocol packet → send → receive reply → verify and unpack.

Course arrangement note: This chapter places Socket custom protocols after the TCP tutorial because the TCP tutorial has already fully covered Socket programming basics (`socket()`, `connect()`, `send()`, `recv()`, `close()`). This experiment won't repeat these basic API explanations but focuses on higher-level skills—**how to design binary protocol struct, how to calculate checksum, how to pack and unpack**. The TCP tutorial solves "how to transmit", this experiment solves "what to transmit, how to verify what was transmitted".

2. Core Concepts

2.1 Custom Protocol Packet Structure

The protocol packet consists of **header (8 bytes) + payload (max 64 bytes)**. Header is fixed at 8 bytes, actual frame length = 8 + `length`, ultimately no more than 72 bytes.

header	version	type	length	checksum	data[64]
1B	1B	2B	2B	2B	max 64B
0xAA	0x01				

Header 8 Bytes

Payload = length bytes

Field	Offset	Size	Description
header	0	1B	Frame header, fixed 0xAA, first verification step when receiving
version	1	1B	Protocol version number
type	2	2B	Message type (big-endian), e.g. 0x0001 = query request
length	4	2B	Payload byte count (big-endian), max 64
checksum	6	2B	ICMP-style checksum of header + payload (big-endian)
data	8	0~64B	Payload data, empty when length is 0

C struct definition (`__attribute__((packed))` forbids compiler padding/alignment bytes):

```
typedef struct {
    uint8_t header;    // Frame header 0xAA
    uint8_t version;   // Version number
    uint16_t type;      // Message type (network byte order)
    uint16_t length;    // Data length (network byte order)
    uint16_t checksum;  // Checksum (network byte order)
    char data[64];     // Payload data
} __attribute__((packed)) proto_pkt_t;
```

Pack Example: "Hello YAHBOOM" (13-byte payload)

Sending "Hello YAHBOOM" (13 bytes), complete protocol packet is **21 bytes**, hex breakdown:

AA 01 00 01 00 0D F4 2A 48 65 6C 6C 6F 20 59 41 48 42 4F 4F 4D

type

length

checksum

data

2B BE

2B BE

LE 2B

13B ASCII

version

0x01

header

0xAA

Offset	Byte	Field	Value	Meaning
0	AA	header	0xAA	Frame header identifier
1	01	version	1	Protocol version
2-3	00 01	type	0x0001	Query request (big-endian)
4-5	00 0D	length	13	Payload 13 bytes (big-endian)
6-7	F4 2A	checksum	0xF42A	Checksum (ESP32 little-endian, ntohs displays)

Offset	Byte	Field	Value	Meaning
8-20	48 65 ... 4D	data	"Hello YAHBOOM"	Payload ASCII

Key points:

- `length` field determines actual frame length = **8 + length**. ESP32 receives according to this byte count, stops when full
- `data[64]` is max capacity, actually only uses first `length` bytes, remaining part is auto-zeroed by `memset`
- When sending from network debugging assistant, only fill to `8 + length` bytes, no need to pad to 72
- `checksum` coverage = 8-byte header + `length` bytes payload, does not include padding zeros

2.2 length Field Calculation

The `length` field stores the actual byte count of payload `data[]` (excluding 8-byte header). Obtained via `strlen()` then converted to big-endian:

```
strcpy(pkt.data, "Hello YAHBOOM");
pkt.length = htons((uint16_t)strlen(pkt.data)); // length = 0x000D (13, big-endian)
```

Whatever `length` the sender fills, the receiver receives exactly `8 + length` bytes and stops when full. Max is limited by `data[64]` to 64 bytes, but sender and receiver negotiate actual frame length via `length`—no need to send full.

2.3 Checksum Calculation (ICMP Style)

Checksum detects bit errors in data transmission, coverage = **8-byte header + length bytes payload**.

Calculation Steps (ESP32 Little-Endian View)

Example with "Hello YAHBOOM", 21 bytes of raw data (checksum position filled with 00 00 first):

```
AA 01 00 01 00 0D 00 00 48 65 6C 6C 6F 20 59 41 48 42 4F 4F 4D
```

Important: ESP32 is a little-endian CPU. `calc_checksum((uint16_t*)&pkt, len)` reads each pair of bytes in **little-endian order**. That is, memory `AA 01` is read as `uint16_t` value `0x01AA` (low byte first).

Step 1: Little-endian 16-bit grouped accumulation

```
0x01AA + 0x0100 + 0x0D00 + 0x0000 + 0x6548 + 0x6C6C + 0x206F + 0x4159 + 0x4248 + 0x4F4F + 0x004D
```

↑
odd byte, high 8

```
bits padded with 0  
= 0x1D50A
```

Step 2: Carry fold (twice)

```
0x1D50A → sum = (sum >> 16) + (sum & 0xFFFF)
         = 0x0001 + 0xD50A
         = 0xD50B
```

```
0xD50B → sum = sum + (sum >> 16)
        = 0xD50B + 0x0000
        = 0xD50B
```

Step 3: Bitwise NOT

```
~0xD50B = 0x2AF4
```

Checksum host order value = 0x2AF4, stored in memory in **little-endian** = low byte first → F4 2A. Log ntohs(checksum) shows 0xF42A.

C Code Implementation

```
static uint16_t calc_checksum(uint16_t *buf, int len)
{
    uint32_t sum = 0;
    while (len > 1) { sum += *buf++; len -= 2; }
    if (len == 1) sum += *(uint8_t *)buf;
    sum = (sum >> 16) + (sum & 0xffff);
    sum += sum >> 16;
    return (uint16_t)(~sum);
}
```

Pack Call Flow

```
proto_pkt_t pkt;
memset(&pkt, 0, sizeof(pkt));
pkt.header = 0xAA;
pkt.version = 1;
pkt.type = htons(0x0001);
strcpy(pkt.data, "Hello YAHBOOM");
pkt.length = htons((uint16_t)strlen(pkt.data));
pkt.checksum = 0;
pkt.checksum = calc_checksum((uint16_t *)&pkt,
                             8 + ntohs(pkt.length));
```

Receive Packet Verification Flow

Save received checksum → Clear checksum field → Recalculate calc_checksum() for (8 + length) bytes
→ Compare two checksums → Match = verified, mismatch = data corrupted

2.4 Network Byte Order

Multi-byte fields (`type`, `length`, `checksum`) use **big-endian (network byte order)** uniformly on the network:

```
pkt.type    = htons(0x0001);    // Host order → Network order
pkt.length  = htons(strlen(data));

// Receiver converts back to host order
uint16_t type = ntohs(pkt.type);
```

3. Project Architecture and Flow

```
app_main()
├─ wifi_init()
│   ├─ nvs_flash_init()
│   ├─ esp_netif_init()
│   ├─ esp_event_loop_create_default()
│   ├─ xEventGroupCreate()
│   ├─ esp_netif_create_default_wifi_sta()
│   ├─ esp_wifi_init() + register event callback
│   └─ esp_wifi_set_mode(STA) + esp_wifi_start()
│
├─ wifi_connect()
│   └─ wait for WiFi to get IP (WIFI_OK, 30s timeout)
│       └─ wifi_handler():
│           ├─ WIFI_EVENT_STA_START → esp_wifi_scan_start()    Scan APs
│           ├─ WIFI_EVENT_SCAN_DONE → Print AP list → Connect to target WiFi
│           ├─ WIFI_EVENT_STA_DISCONNECTED → Auto reconnect
│           └─ IP_EVENT_STA_GOT_IP → Set WIFI_OK, print IP + gateway
│
└─ socket_experiment()
    ├─ 1. socket(AF_INET, SOCK_STREAM, IPPROTO_TCP) → Create TCP socket
    ├─ 2. connect(192.168.11.183:8080) → Three-way handshake
    ├─ 3. Build proto_pkt_t struct (fill fields → clear checksum → calculate & write)
    ├─ 4. send() send protocol packet (8 + length bytes)
    └─ 5. while(1) loop receive packets → header verify → checksum unpack → continue
```

4. Hardware Dependencies

- **Hardware Connection:** No external wiring, WiFi wireless communication
- **Required Components:** `lwip/sockets`, `esp_wifi`, `esp_netif`, `esp_event`, `nvs_flash`, `FreeRTOS`

5. Source Code Analysis

Full source: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `5.Socket`

5.1 Header Files and Macro Configuration

This experiment introduces LWIP Socket API, ESP32 WiFi driver, and FreeRTOS event groups. Defines three macros for WiFi connection, server address, and max scan count. WiFi hotspot and server IP/port can be modified per actual environment.

```
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "lwip/sockets.h"
#include "esp_log.h"
#include "esp_netif.h"
#include "esp_event.h"
#include "esp_wifi.h"
#include "nvs_flash.h"

static const char *TAG = "SOCKET";
#define WIFI_SSID      "Your_WiFi_SSID"
#define WIFI_PASS      "Your_WiFi_Password"
// Server IP
#define SERVER_IP      "192.168.11.183"
// Server port
#define SERVER_PORT     8080
// Max APs to scan
#define MAX_SCAN       32
```

5.2 WiFi Initialization and AP Scan

Terminology distinction: In this section, "AP" refers to **Wireless Access Point**—the WiFi router that ESP32 as STA (client) connects to, **not** ESP32's AP mode (self-created hotspot). This experiment runs entirely in **STA mode**.

This experiment extends the basic STA WiFi connection with **AP scanning**—before connecting, scan surrounding WiFi and print AP list (SSID, RSSI, channel), convenient for on-site debugging to confirm signal environment and channel usage. Basic WiFi STA connection flow (NVS init, event callback, EventGroup sync, etc.) is identical to **5.2 WIFI-STA Mode**, only the new scan logic is shown here.

Basic review: 5. Network Protocols and Wireless Communication -> 2. WIFI-STA Mode -> WIFI-STA Mode

5.3 wifi_handler() — With AP Scan

This experiment's WiFi callback has one more function than basic STA: before connecting, **scan surrounding WiFi**, print AP list (SSID, signal strength RSSI, channel), then connect to target router. This is very useful when debugging WiFi connection issues on-site—you can see what hotspots are around, if the signal is strong enough, if there's co-channel interference.

Flow: STA starts → `esp_wifi_scan_start()` scan → `SCAN_DONE` event triggers → `esp_wifi_scan_get_ap_records()` get records → iterate and print → configure target AP → `esp_wifi_connect()`.

```

static void wifi_handler(void *arg, esp_event_base_t base, int32_t id, void *data)
{
    if (base == WIFI_EVENT && id == WIFI_EVENT_STA_START) {
        esp_wifi_scan_start(NULL, false);
    }
    else if (base == WIFI_EVENT && id == WIFI_EVENT_SCAN_DONE) {
        uint16_t n = 0;
        esp_wifi_scan_get_ap_num(&n);
        if (n > 0) {
            uint16_t cnt = (n > MAX_SCAN) ? MAX_SCAN : n;
            wifi_ap_record_t *aps = calloc(cnt, sizeof(wifi_ap_record_t));
            esp_wifi_scan_get_ap_records(&cnt, aps);
            ESP_LOGI(TAG, "==== Found %d APs ====", cnt);
            for (int i = 0; i < cnt; i++) {
                ESP_LOGI(TAG, "[%2d] SSID: %-32s RSSI: %4d CH: %d",
                    i + 1, aps[i].ssid, aps[i].rssi, aps[i].primary);
            }
            free(aps);
        }
        wifi_config_t cfg = {0};
        strcpy((char *)cfg.sta.ssid, WIFI_SSID);
        strcpy((char *)cfg.sta.password, WIFI_PASS);
        esp_wifi_set_config(WIFI_IF_STA, &cfg);
        esp_wifi_connect();
    }
    else if (base == WIFI_EVENT && id == WIFI_EVENT_STA_CONNECTED) {
    }
    else if (base == WIFI_EVENT && id == WIFI_EVENT_STA_DISCONNECTED) {
        wifi_event_sta_disconnected_t *e = data;
        ESP_LOGE(TAG, ">>> Disconnected! reason=%u reconnecting... <<<", e->reason);
        esp_wifi_connect();
    }
    else if (base == IP_EVENT && id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *e = data;
        ESP_LOGI(TAG, "Got IP: " IPSTR " Gateway: " IPSTR,
            IP2STR(&e->ip_info.ip), IP2STR(&e->ip_info.gw));
        xEventGroupSetBits(wifi_evt, WIFI_OK);
    }
}

```

5.4 socket_experiment() — Pack/Send/Receive/Verify

All new logic in this experiment is encapsulated in this function: create Socket → connect to server → pack → send → **loop receive and verify**. Receiving uses `while(1)` infinite loop, **network debugging assistant sends one frame, ESP32 receives and parses immediately**, sends ten frames and parses ten times, doesn't disconnect.

Key for packing: first `memset` to clear the entire struct—if not cleared, uninitialized fields may be random garbage data and checksum won't match. Then fill fields one by one, `header` fixed to `0xAA` as frame header, `type` and `length` use `htons()` to convert to network byte order. Clear the checksum field itself first (calculation covers entire struct, doesn't include itself), then call `calc_checksum()` to calculate and fill back.

After receiving, unpack and verify (executed for each frame received in the loop): first save received checksum → clear checksum field in struct → recalculate checksum → compare with saved value. If same, data was not corrupted during transmission. Frames with header not equal to `0xAA` and checksum mismatch are skipped by `continue`, continuing to wait for next frame.

```
static void socket_experiment(void)
{
    /* 1. Create socket */
    int sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (sock < 0) {
        ESP_LOGE(TAG, "1. socket() failed, errno=%d", errno);
        return;
    }
    ESP_LOGI(TAG, "1. socket() created OK (fd=%d)", sock);

    /* 2. Connect to server */
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(SERVER_PORT);
    inet_pton(AF_INET, SERVER_IP, &addr.sin_addr);

    ESP_LOGI(TAG, "2. connect() -> %s:%d ...", SERVER_IP, SERVER_PORT);
    if (connect(sock, (struct sockaddr *)&addr, sizeof(addr)) != 0) {
        ESP_LOGE(TAG, "2. connect() failed! errno=%d", errno);
        close(sock);
        return;
    }
    ESP_LOGI(TAG, "2. connect() OK, TCP link established");

    /* ===== 3. Build packet ===== */
    proto_pkt_t pkt;
    memset(&pkt, 0, sizeof(pkt));

    pkt.header = 0xAA;
    pkt.version = 1;
    pkt.type = htons(0x0001);
    strcpy(pkt.data, "ESP32-TCP-PACKET");
    pkt.length = htons((uint16_t)strlen(pkt.data));
    pkt.checksum = 0;
    pkt.checksum = calc_checksum((uint16_t *)&pkt, sizeof(pkt));

    ESP_LOGI(TAG, "3. Packet built OK!");
    ESP_LOGI(TAG, "    header = 0x%02X", pkt.header);
    ESP_LOGI(TAG, "    version = %u", pkt.version);
    ESP_LOGI(TAG, "    type = 0x%04X (net)", ntohs(pkt.type));
    ESP_LOGI(TAG, "    length = %u (net)", ntohs(pkt.length));
    ESP_LOGI(TAG, "    data = \"%s\"", pkt.data);
    ESP_LOGI(TAG, "    checksum = 0x%04X (net)", ntohs(pkt.checksum));
    ESP_LOGI(TAG, "    sizeof(pkt) = %u bytes", sizeof(pkt));

    /* ===== 4. Send packet ===== */
    int sent = send(sock, &pkt, sizeof(pkt), 0);
```

```

if (sent > 0) {
    ESP_LOGI(TAG, "4. Send OK! sent %d/%u bytes", sent, sizeof(pkt));
} else {
    ESP_LOGE(TAG, "4. Send failed! errno=%d", errno);
    close(sock);
    return;
}

/* ===== 5. Send/receive loop ===== */
while (1) {
    proto_pkt_t rsp;
    memset(&rsp, 0, sizeof(rsp));

    // Receive 8-byte header
    int total = 0;
    while (total < 8) {
        int n = recv(sock, (uint8_t *)&rsp + total, 8 - total, 0);
        if (n <= 0) break;
        total += n;
    }
    if (total < 8) {
        vTaskDelay(pdMS_TO_TICKS(100));
        continue;
    }

    // Parse length, clamp to prevent overflow
    uint16_t body_len = ntohs(rsp.length);
    if (body_len > sizeof(rsp.data)) body_len = sizeof(rsp.data);
    int need = 8 + body_len;

    // Receive remaining payload
    while (total < need) {
        int n = recv(sock, (uint8_t *)&rsp + total, need - total, 0);
        if (n <= 0) break;
        total += n;
    }
    if (total < need) {
        vTaskDelay(pdMS_TO_TICKS(100));
        continue;
    }

    // Print packet fields
    ESP_LOGI(TAG, "5. Recv OK! %d bytes (need=%d)", total, need);
    ESP_LOGI(TAG, "    header    = 0x%02X", rsp.header);
    ESP_LOGI(TAG, "    version  = %u",    rsp.version);
    ESP_LOGI(TAG, "    type     = 0x%04X", ntohs(rsp.type));
    ESP_LOGI(TAG, "    length   = %u",    body_len);
    ESP_LOGI(TAG, "    checksum = 0x%04X (rx)", ntohs(rsp.checksum));
    ESP_LOGI(TAG, "    data     = \"%s\"", (int)body_len, rsp.data);

    // Frame header check

```

```

    if (rsp.header != 0xAA) {
        ESP_LOGW(TAG, "    Bad header! expected 0xAA");
        ESP_LOG_BUFFER_HEX(TAG, &rsp, total);
        continue;
    }

    // Checksum verification
    uint16_t rx_cksum = rsp.checksum;
    rsp.checksum = 0;
    uint16_t calc = calc_checksum((uint16_t *)&rsp, total);
    if (rx_cksum == calc) {
        ESP_LOGI(TAG, "    Checksum OK! (0x%04x == 0x%04x)",
            ntohs(rx_cksum), ntohs(calc));
    } else {
        ESP_LOGW(TAG, "    Checksum FAIL! rx=0x%04x calc=0x%04x",
            ntohs(rx_cksum), ntohs(calc));
        ESP_LOG_BUFFER_HEX(TAG, &rsp, total);
    }
}
}

### 5.5 app_main() – Entry Point

```c
void app_main(void)
{
 wifi_init();
 wifi_connect();
 socket_experiment();
}

```

## 5.6 Key Points of Pack Flow

```

Clear struct → Fill header → Fill version → Fill type (htons) → Fill data
→ strlen calculate length → Fill length (htons) → checksum = 0 → calc_checksum(&pkt, 72)
→ Return value written back to pkt.checksum

```

## 6. Operation Flow

### 6.1 Build and Flash Flow

**ESP-IDF Flash Flow:** See `ESP-IDF → 1. Before Development → 3. Build and Flash`.

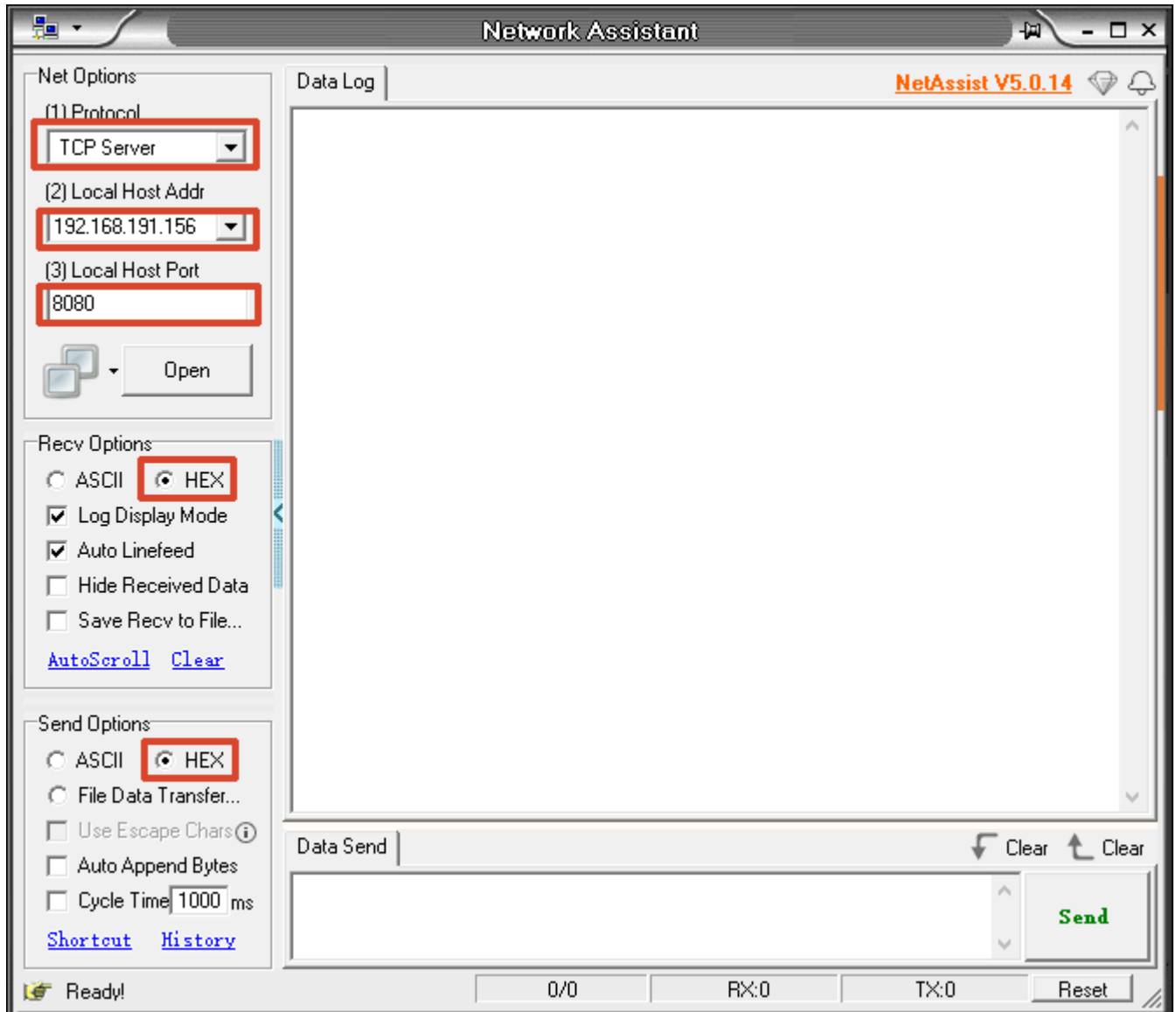
### 6.2 Network Debugging Assistant Configuration

This experiment requires a TCP server on PC to receive binary protocol packets from ESP32 and reply.

Recommended to use **NetAssist (Network Debugging Assistant)**, steps:

1. **Open** NetAssist, protocol type on left select **TCP Server**
2. **Set port:** local port fill `8080` (consistent with `SERVER_PORT` in code), click "Start Listening"

3. **Receive settings:** Must check **HEX Display** (hexadecimal), otherwise binary frame header `0xAA`, checksum and other fields will display as garbled text
4. **Send settings:** Check **HEX Send**, reply data must also be in this protocol format (frame header `AA` + version `01` + type `00 01` + length + checksum + data), otherwise ESP32 checksum unpack will fail
5. **Confirm network:** Ensure PC and ESP32 are connected to the same router (same subnet), and PC firewall allows port 8080 inbound

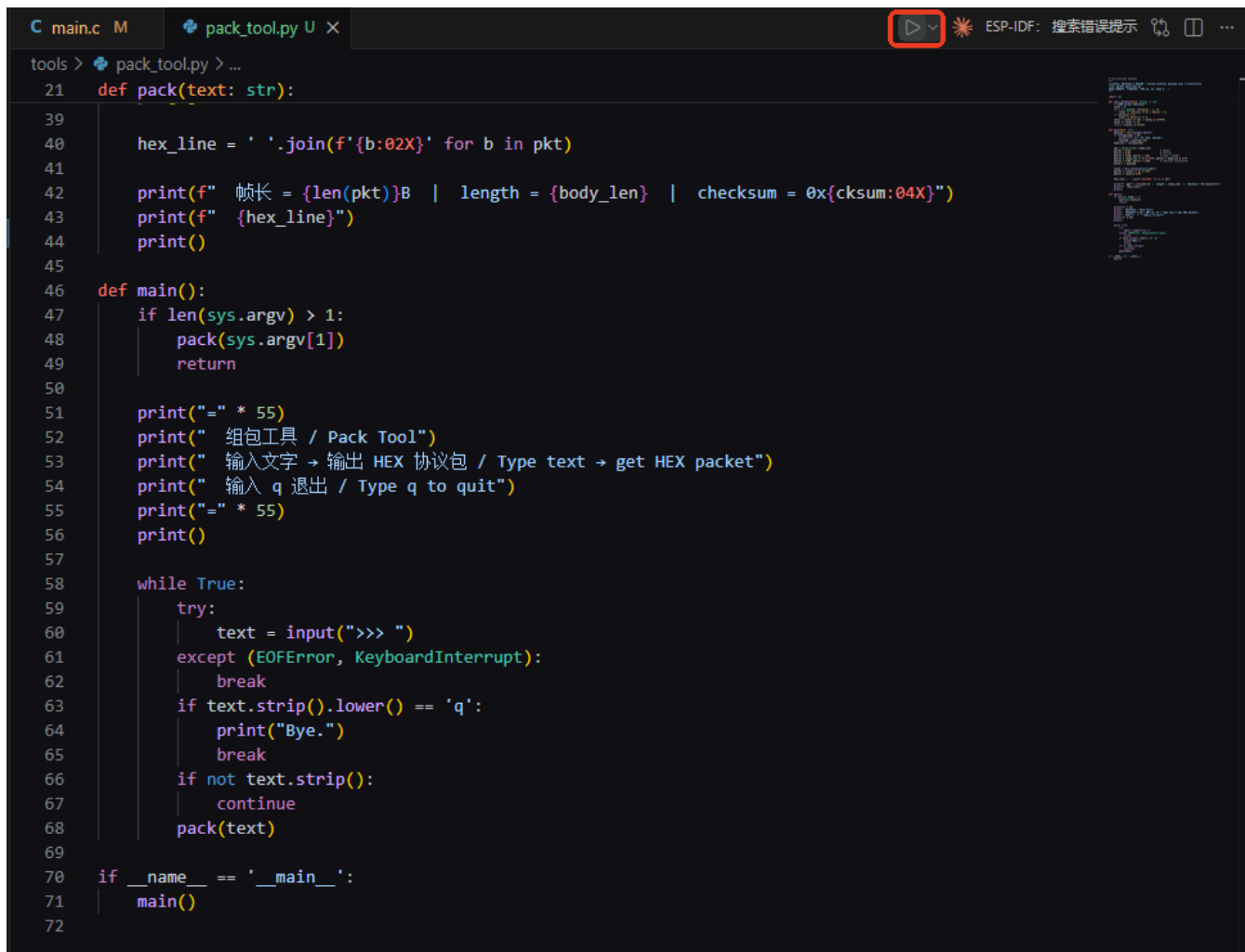


If NetAssist receives data but ESP32 logs show `Recv timeout/no reply`, the server is not replying. Check HEX send, reply in protocol format.

## 6.3 Pack Tool

Manual packing easily fills wrong `length` and `checksum`. Recommend using the accompanying `pack_tool.py` script:

1. Open terminal, go to `tools/` directory
2. Run `python pack_tool.py`, enter interactive mode
- 3.



```
21 def pack(text: str):
39
40 hex_line = ' '.join(f'{b:02X}' for b in pkt)
41
42 print(f" 帧长 = {len(pkt)}B | length = {body_len} | checksum = 0x{cksum:04X}")
43 print(f" {hex_line}")
44 print()
45
46 def main():
47 if len(sys.argv) > 1:
48 pack(sys.argv[1])
49 return
50
51 print("=" * 55)
52 print(" 组包工具 / Pack Tool")
53 print(" 输入文字 → 输出 HEX 协议包 / Type text → get HEX packet")
54 print(" 输入 q 退出 / Type q to quit")
55 print("=" * 55)
56 print()
57
58 while True:
59 try:
60 text = input(">>> ")
61 except (EOFError, KeyboardInterrupt):
62 break
63 if text.strip().lower() == 'q':
64 print("Bye.")
65 break
66 if not text.strip():
67 continue
68 pack(text)
69
70 if __name__ == '__main__':
71 main()
72
```

4. Enter any text (Chinese or English), press Enter to output HEX protocol packet
5. Copy output hex string, paste into network debugging assistant HEX send box
6. Enter `q` to quit

```
=====
Pack Tool
Input text → Output HEX packet
Input q to quit
=====

>>> Hello YAHBOOM
Frame len = 21B | length = 13 | checksum = 0xF42A
AA 01 00 01 00 0D F4 2A 48 65 6C 6C 6F 20 59 41 48 42 4F 4F 4D

>>> Hello ESP32
Frame len = 19B | length = 11 | checksum = 0x630A
AA 01 00 01 00 0B 63 0A E4 BD A0 E5 A5 BD 45 53 50 33 32

>>> q
Bye.
```

```
=====
组包工具 / Pack Tool
输入文字 → 输出 HEX 协议包 / Type text → get HEX packet
输入 q 退出 / Type q to quit
=====

>>> Hello YAHBOOM
帧长 = 21B | length = 13 | checksum = 0xF42A
AA 01 00 01 00 0D F4 2A 48 65 6C 6C 6F 20 59 41 48 42 4F 4F 4D

>>> 你好ESP32
帧长 = 19B | length = 11 | checksum = 0x630A
AA 01 00 01 00 0B 63 0A E4 BD A0 E5 A5 BD 45 53 50 33 32

>>> |
```

**Core point:** Frame length = 8 + length, automatically changes with input content, **don't** fixed fill 72 bytes or manually pad zeros. `length` and `checksum` are both automatically calculated by the script, no manual calculation needed.

## 6.4 Normal Log and Step Breakdown

After flashing, open serial monitor (baud rate 115200). Normal operation shows these stages in order:

### Stage 1: Scan nearby APs and connect

```
I (2962) SOCKET: ===== Found 10 APs =====
I (2962) SOCKET: [1] SSID: Yahboom2 RSSI: -20 CH: 6
I (2962) SOCKET: [2] SSID: Yahboom3 RSSI: -27 CH: 1
...
```

```
I (2952) SOCKET: ===== Found 10 APs =====
I (2962) SOCKET: [1] SSID: Yahboom2 RSSI: -14 CH: 6
I (2962) SOCKET: [2] SSID: NX30PRO_2.4G RSSI: -35 CH: 6
I (2962) SOCKET: [3] SSID: Yahboom8 RSSI: -38 CH: 6
I (2972) SOCKET: [4] SSID: Yahboom3 RSSI: -39 CH: 1
I (2982) SOCKET: [5] SSID: Yahboom RSSI: -49 CH: 11
I (2982) SOCKET: [6] SSID: Yahboom RSSI: -51 CH: 1
I (2992) SOCKET: [7] SSID: Yahboom_ESP32_WIFI RSSI: -56 CH: 6
I (3002) SOCKET: [8] SSID: Yahboom_ESP32_WIFI RSSI: -58 CH: 6
I (3002) SOCKET: [9] SSID: Yahboom RSSI: -61 CH: 6
I (3012) SOCKET: [10] SSID: Yahboom RSSI: -61 CH: 6
```

ESP32 starts STA and automatically scans nearby WiFi, printing each AP's **SSID (name)**, **RSSI (signal strength, closer to 0 is stronger)**, **CH (channel)**. Then automatically connects to hotspot specified by `WIFI_SSID`.

### Stage 2: Get IP address

After WiFi link is established, DHCP automatically assigns IP. `Got IP` shows local IP and gateway address. Network ready, can establish Socket connection.

```
I (4362) esp_netif_handlers: sta ip: 192.168.11.181, mask: 255.255.255.0, gw: 192.168.11.1
I (4362) SOCKET: Got IP: 192.168.11.181 Gateway: 192.168.11.1
I (4362) SOCKET: WiFi connected successfully!
```

```
I (4162) esp_netif_handlers: sta ip: 192.168.11.181, mask: 255.255.255.0, gw: 192.168.11.1
I (4162) SOCKET: Got IP: 192.168.11.181 Gateway: 192.168.11.1
I (4162) SOCKET: WiFi connected successfully!
```

### Stage 3: Establish TCP connection

```
I (4162) SOCKET: 1. socket() created OK (fd=54)
I (4172) SOCKET: 2. connect() -> 192.168.11.183:8080 ...
I (4192) wifi:<ba-addr>idx:1 (ifx:0, 80:8f:1d:20:9d:f7), tid:0, ssn:0, winSize:64
I (4202) SOCKET: 2. connect() OK, TCP link established
```

```
I (4162) SOCKET: 1. socket() created OK (fd=54)
I (4172) SOCKET: 2. connect() -> 192.168.11.183:8080 ...
I (4192) wifi:<ba-addr>idx:1 (ifx:0, 80:8f:1d:20:9d:f7), tid:0, ssn:0, winSize:64
I (4202) SOCKET: 2. connect() OK, TCP link established
```

Step 1 creates IPv4 TCP socket (returns fd=54), step 2 completes three-way handshake with server 192.168.11.183:8080.

### Stage 4: Pack and send

In `app_main()`, ESP32 calls `wifi_init()` → `wifi_connect()` → `socket_experiment()` in sequence. `socket_experiment()` first completes Socket connection, then `memset` clears struct, fills fields, calculates checksum, finally `send()`.

```
I (3864) SOCKET: 3. Packet built OK!
I (3864) SOCKET: header = 0xAA
I (3865) SOCKET: version = 1
I (3865) SOCKET: type = 0x0001 (net)
I (3865) SOCKET: length = 16 (net)
I (3865) SOCKET: data = "ESP32-TCP-PACKET"
I (3866) SOCKET: checksum = 0x8F3A (net)
I (3866) SOCKET: sizeof(pkt) = 72 bytes
I (3868) SOCKET: 4. Send OK! sent 72/72 bytes
```

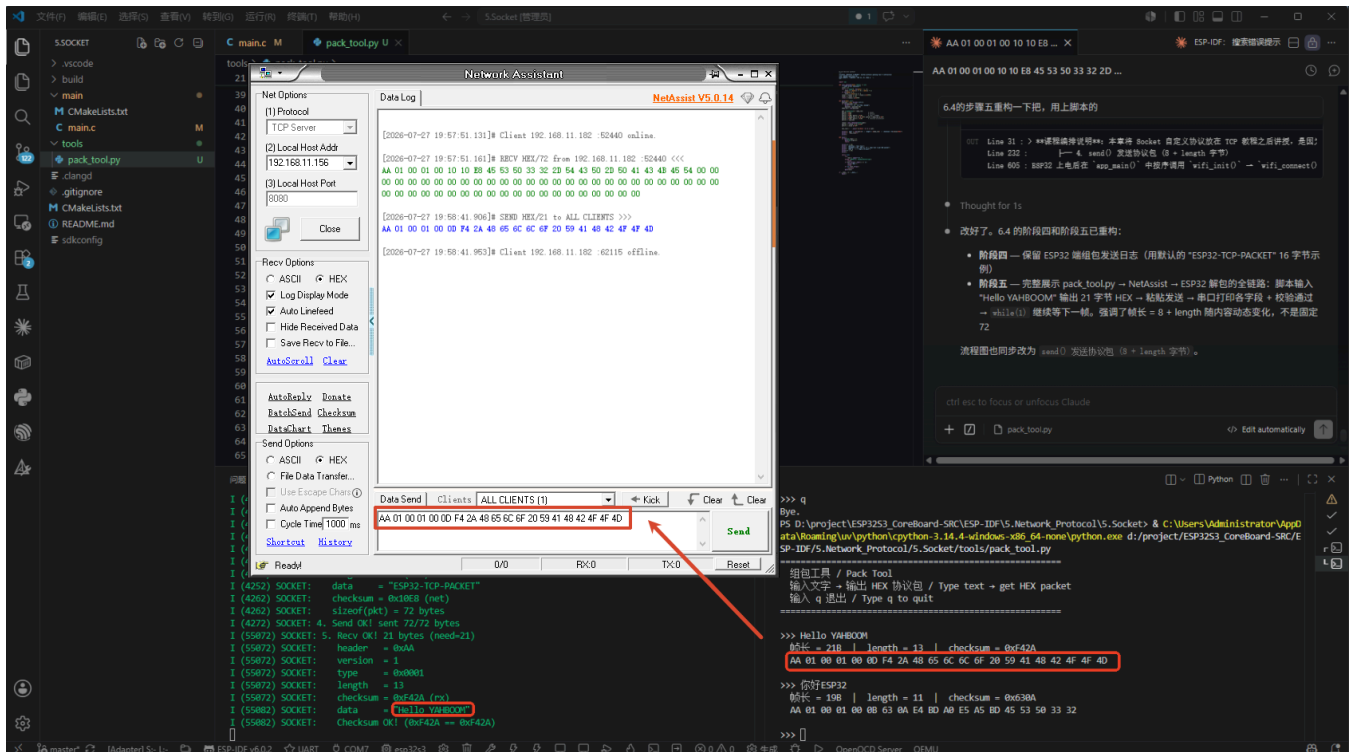
```
I (4202) SOCKET: 3. Packet built OK!
I (4202) SOCKET: header = 0xAA
I (4202) SOCKET: version = 1
I (4202) SOCKET: type = 0x0001 (net)
I (4212) SOCKET: length = 16 (net)
I (4212) SOCKET: data = "ESP32-TCP-PACKET"
I (4222) SOCKET: checksum = 0x10E8 (net)
I (4222) SOCKET: sizeof(pkt) = 72 bytes
I (4232) SOCKET: 4. Send OK! sent 72/72 bytes
```

### Stage 5: Loop receive and unpack (network debugging assistant sends packet)

After ESP32 sends, enters `while(1)` loop to wait. Switch to **pack\_tool.py**, input text to get HEX protocol packet, paste into NetAssist HEX send box and send.

```
>>> Hello YAHBOOM
Frame len = 21B | length = 13 | checksum = 0xF42A
AA 01 00 01 00 0D F4 2A 48 65 6C 6C 6F 20 59 41 48 42 4F 4F 4D
```

Paste the script's output hex string into NetAssist, check HEX send, click send:



ESP32 receives and **parses each frame independently**, serial output:

```
I (14692) SOCKET: 5. Recv OK! 21 bytes (need=21)
I (14692) SOCKET: header = 0xAA
I (14692) SOCKET: version = 1
I (14702) SOCKET: type = 0x0001
I (14702) SOCKET: length = 13
I (14702) SOCKET: checksum = 0x2AF4 (rx)
I (14702) SOCKET: data = "Hello YAHBOOM"
I (14712) SOCKET: Checksum OK! (0x2AF4 == 0x2AF4)
```

Parse flow: first `recv()` loop to receive full 8-byte header → read `length` → continue receiving `length` bytes payload → print each field → header verify `0xAA` → checksum verification. Whether checksum passes, header mismatch, or checksum failure, all continue waiting for next frame.

```
I (55072) SOCKET: 5. Recv OK! 21 bytes (need=21)
I (55072) SOCKET: header = 0xAA
I (55072) SOCKET: version = 1
I (55072) SOCKET: type = 0x0001
I (55072) SOCKET: length = 13
I (55072) SOCKET: checksum = 0xF42A (rx)
I (55082) SOCKET: data = "Hello YAHBOOM"
I (55082) SOCKET: Checksum OK! (0xF42A == 0xF42A)
```

**Key point:** Frame length is `8 + length` dynamically changing. "Hello YAHBOOM" frame length is 21 bytes, "ESP32-TCP-PACKET" frame length is 24 bytes, not fixed 72. **Each frame received and immediately parsed and output.** Can continuously send different content and different length protocol packets, send ten frames and unpack ten times, doesn't disconnect.

## 7. Common Problems

Symptom	Cause	Solution
Checksum mismatch	Sender and receiver checksum calculation range inconsistent	Confirm both sides clear checksum field before calculation
<code>sizeof(pkt)</code> is not 72	Compiler padded alignment bytes	Must add <code>__attribute__((packed))</code>
Receive only partial data	TCP byte stream fragmentation	Loop <code>recv()</code> until full <code>sizeof(proto_pkt_t)</code> bytes received
Multi-byte field values abnormal on network	Forgot byte order conversion	Use <code>htons()</code> for packing <code>uint16_t</code> fields, <code>ntohs()</code> for unpacking
Scan AP list empty	WiFi mode not set to STA	<code>esp_wifi_set_mode(WIFI_MODE_STA)</code> must be before <code>esp_wifi_start()</code>
<code>recv()</code> timeout or no reply	Server not running or not replying	Ensure PC TCP Server can receive and reply same format struct
Network debugging assistant shows garbled text	Not switched to HEX display mode	Binary protocol packets are not ASCII strings, need to change debug assistant receive display mode to <b>hexadecimal/HEX</b>